



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

• Experiment No.: 7

1. Title:

Perform DOM traversal and updates; develop a to-do list app with add, edit, and delete features.

2. Software/Tools Required:

Visual Studio Code, Web Browser (Google Chrome / Mozilla Firefox), Node.js (for local server execution)

3. Theory:

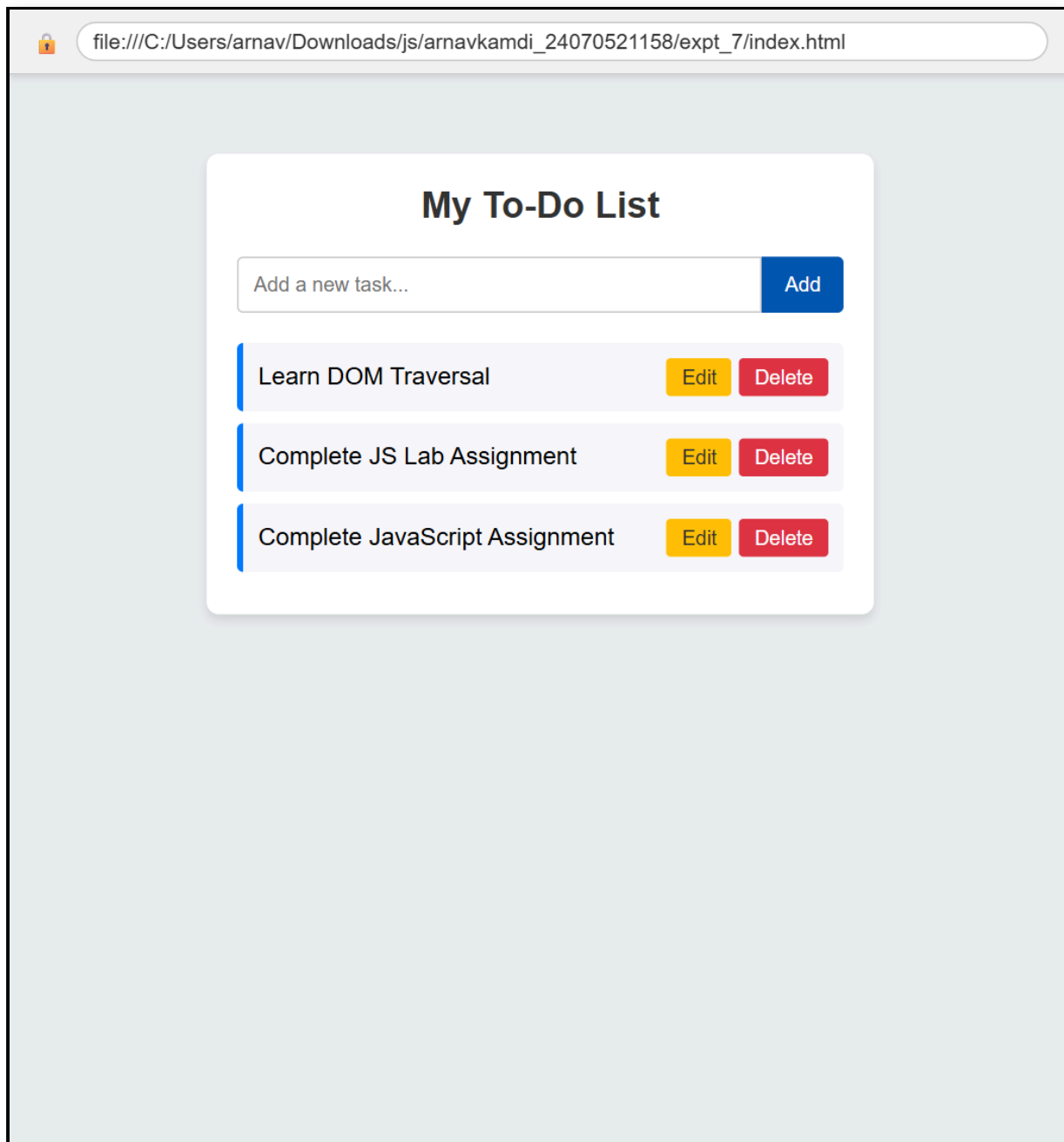
The Document Object Model (DOM) is a programming interface for web documents. It represents the page so that programs can change the document structure, style, and content. The DOM represents the document as nodes and objects; that way, programming languages can interact with the page.

DOM Traversal refers to the process of navigating through this node tree. JavaScript provides properties like `parentNode`, `childNodes`, `firstChild`, and `nextSibling` to move from one node to another relative to their position in the hierarchy. This is crucial when an event occurs on a specific element (like

a click on a "Delete" button) and the script needs to find the corresponding parent element (like the specific list item) to act upon.

DOM Updates involve modifying the structure of the document dynamically. Elements can be created from scratch using `document.createElement()`, populated with text using `innerText`, and inserted into the live document using methods like `appendChild()` or `insertBefore()`. Conversely, elements can be removed using `removeChild()`. Building an interactive To-Do list requires all these concepts: creating new list items when tasks are added, traversing to specific elements when editing, and removing specific nodes from the DOM tree when a task is deleted.

4. Output:



5. Result/Conclusion:

The To-Do list application was successfully developed and functions smoothly. The script correctly handled DOM updates by dynamically creating new task elements and appending them to the list. Event listeners and DOM traversal were effectively used to implement the edit and delete functionalities, allowing the user to manage their tasks seamlessly without needing to reload the webpage.